

스케줄링 : 개요

이제 프로세스를 실행시키기 위한 문맥 교환 등의 저수준 기법에 대해서는 분명하게 이해하고 있어야 한다. 그렇지 않다면 한 두 장 이전으로 돌아가서 이러한 기법들이 어떻게 동작하는지에 대한 설명을 다시 읽기 바란다. 운영체제 스케줄러의 고수준 정책에 관해서는 이해가 필요하다. 이제부터 다양한 **스케줄링 정책(scheduling policy)**을 소개하고 그에 관한 이해를 높일 것이다. 이러한 정책은 **원칙(discipline)**이라고도 불리며 많은 똑똑하고 부지런한 사람들이 오랫동안 개발한 정책들이다.

사실 스케줄링의 기원은 컴퓨터 시스템 개발 이전으로 거슬러 올라간다. 초기의 방법들은 생산 관리 분야에서 개발되었으며 컴퓨터에 적용되었다. 이 사실은 그렇게 놀라운 일은 아니다. 생산 공정과 기타 많은 인간의 행동 역시 스케줄링이 필요하고, 효율성에 대한 요구 등, 공통된 해결 사항이 존재하기 때문이다. 우리의 문제는 다음과 같다.

핵심 질문 : 스케줄링 정책은 어떻게 개발하는가

스케줄링 정책을 생각하기 위한 기본적인 프레임워크를 어떻게 만들어야 하는가? 핵심 가정은 무엇인가? 어떤 평가 기준이 중요한가? 컴퓨터 시스템의 초창기에 사용되었던 기본 접근법은 무엇인가?

10.1 워크로드에 대한 가정

가장 먼저 프로세스에 대하여 몇 가지 가정을 할 것이다. 일련의 프로세스들이 실행하는 상황을 **워크로드(workload)**라고 부르기로 한다. 워크로드를 결정하는 것은 정책 개발에 매우 중요한 부분이다. 워크로드에 관해 더 잘 알수록 그에 맞게 정책을 정교하게 손질할 수 있다.

우리가 지금 설명에서 사용할 워크로드에 대한 가정은 비현실적이다. 하지만, 당장 논의하는 데 아무 문제 없다. 앞으로 논의가 진행됨에 따라 가정을 완화시킬 것이고 최종적으로 제대로 동작하는 스케줄링 정책을 만들게 될 것이다.

우리는 시스템에서 실행 중인 프로세스 혹은 작업(job)에 대해 다음과 같은 가정을 한다.

1. 모든 작업은 같은 시간 동안 실행된다.
2. 모든 작업은 동시에 도착한다.
3. 각 작업은 시작되면 완료될 때까지 실행된다.
4. 모든 작업은 CPU만 사용한다(즉, 입출력을 수행하지 않는다).
5. 각 작업의 실행 시간은 사전에 알려져 있다.

이러한 가정들이 비현실적이라고 이미 언급하였다. Animal Farm [Orw45]에서 어떤 동물은 다른 동물보다 더 평등하다고 한 것처럼 몇몇 가정은 다른 가정에 비해 더 비현실적이다. 특히, 각 작업의 실행 시간이 미리 알려져 있다는 가정은 더욱 그렇다. 이 가정은 스케줄러가 모든 것을 다 알 수 있다는 것을 의미한다. 좋은 일이지는 않지만 가까운 미래에 나올 것 같지는 않다.

10.2 스케줄링 평가 항목

워크로드에 대한 가정 이외에 스케줄링 정책의 비교를 위해 **스케줄링 평가 항목(scheduling metric)**을 결정해야 한다. 스케줄링의 경우 다양한 평가 기준이 존재한다.

우선, 문제를 간단하게 하기 위해 **반환 시간(turnaround time)**이라는 하나의 평가 기준을 사용한다. 작업 반환 시간은 작업이 완료된 시각에서 작업이 시스템에 도착한 시각을 뺀 시간으로 정의된다. 반환 시간 $T_{turnaround}$ 는

$$T_{turnaround} = T_{completion} - T_{arrival} \quad (10.1)$$

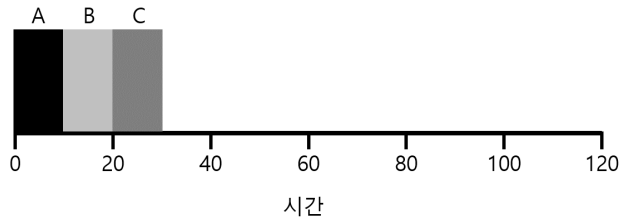
모든 작업은 동시에 도착한다고 가정했으므로 $T_{arrival}=0$ 으로 생각할 수 있다. $T_{turnaround} = T_{completion}$ 이다. 이 가정은 완화해 나갈 것이다.

반환 시간은 **성능** 측면에서의 평가 기준이라는 것을 주목해야 한다. 다른 평가 기준으로 **공정성(fairness)**이다. 예를 들면 **Jain's Fairness Index** [Jai91]에 따라 측정된다. 성능과 공정성은 스케줄링에서는 서로 상충되는 목표이다. 예를 들어, 스케줄러는 성능을 극대화하기 위해 몇몇 작업의 실행을 중지하며, 결과적으로 공정성이 악화된다. 인생은 대부분 완벽하지 않다.

10.3 선입선출

가장 기초적인 알고리즘은 **선입선출(First In First Out, FIFO)** 또는 **선도착선처리(First Come First Served, FCFS)** 스케줄링이라고 알려져 있다. FIFO는 많은 장점을 가진다. 단순하고 구현하기 쉽다. 우리의 가정 하에서는 매우 잘 동작한다.

간단한 예를 스케줄 해 보자. 시스템에 3개의 작업 A, B, C가 거의 동시에 도착했다고 가정하자 ($T_{arrival} = 0$). 간발의 차이로 A, B, C 순서대로 도착했다고 가정하자. 또한, 각 작업은 10초 동안 실행된다고 가정하자. 이 작업들의 평균 반환 시간은 얼마인가?



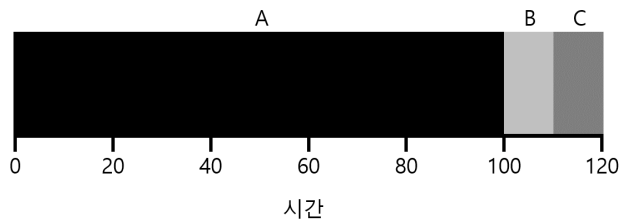
〈그림 10.1〉 FIFO 간단한 예

그림 10.1에서 A는 10, B는 20, C는 30에 종료했다는 것을 알 수 있다. 세 작업의 평균 반환 시간은 $\frac{(10+20+30)}{3} = 20$ 이다. 반환 시간의 계산은 쉽다.

자 이제 가정 중 한 가지를 완화하자. 구체적으로 1번 가정을 완화하여 작업을 실행 시간이 모두 같지 않다고 가정하자. FIFO는 어떻게 동작하는가? FIFO는 어떤 워크로드에서 좋지 않은 성능을 보일까?

(다음으로 넘어가기 전에 생각해 보자... 생각해 생각... 알았지?!)

FIFO 스케줄링이 어떤 문제를 야기하는지 같이 생각해 보자. 구체적으로 다시 A, B, C 세 개의 작업을 가정하자. 그러나 이번에는 A는 100초, B와 C는 10초 동안 실행된다.



〈그림 10.2〉 FIFO가 그렇게 좋은 스케줄링이 아닌 이유

그림 10.2에서 볼 수 있듯이, 작업 A가 B와 C 보다 먼저 100초 동안 실행된다. 따라서 시스템의 평균 반환 시간은 110초로 늘어난다. $\frac{(100+110+120)}{3} = 110$.

이 문제점은 **convoy effect** [Bla+79]라고 칭해지며 짧은 시간 동안 자원을 사용할 프로세스들이 자원을 오랫동안 사용하는 프로세스의 종료를 기다리는 현상을 말한다. 이 스케줄링 시나리오는 슈퍼마켓에서 줄 서서 계산을 기다릴 때, 앞사람이 카트 세 개에 물건을 가득 싣고 있는 경우와 느낌이 비슷하다. 한참 걸리겠지?¹

그러면 어떻게 해야 할까? 작업 실행 시간이 다른 경우 더 좋은 알고리즘은 무엇인가? 먼저 생각해 보아라. 그런 후 다음 내용을 읽기 바란다.

1) 이 경우 빨리 다른 줄에 가서 서거나 길고 깊게 심호흡을 하라고 권하고 싶다. 오케이, 들이쉬고 내쉬고.

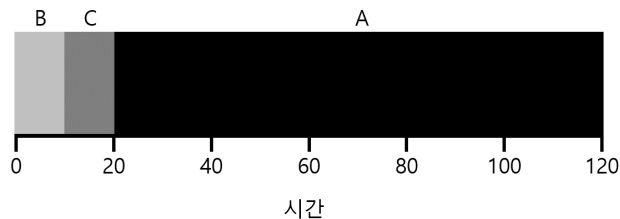
팁: SJF의 원칙

최단 작업 우선 원칙은 평균 반환 시간이 (혹은 우리의 경우, 작업당) 강조되는 모든 시스템에 적용될 수 있는 일반적인 스케줄링 원리이다. 당신이 기다린 경험이 있는 모든 줄을 생각해 보라. 문제의 장소가 고객 만족을 중요시한다면 SJF의 사용을 고려했을 가능성이 높다. 예를 들어, 식료품 가게들은 적은 개수의 물건을 구매한 사람들이 다가올 겨울을 준비하는 가족 뒤에 서서 마냥 기다리지 않도록 “소량 구매 계산대”를 갖추고 있다.

10.4 최단 작업 우선

이 문제는 간단히 해결할 수 있다. 사실 이 아이디어는 오퍼레이션 리서치 분야에서 차용하여 [Cob54; JV56] 컴퓨터 시스템의 작업 스케줄링에 적용한 것이다. 이 새로운 스케줄링 개념용 **최단 작업 우선(Shortest Job First, SJF)**으로 이름 자체가 정책을 설명하고 있기 때문에 기억하기 쉽다. 이 원칙은 가장 짧은 실행 시간을 가진 작업을 먼저 실행시킨다.

위의 예를 이번에는 SJF로 스케줄 해 보자. 그림 10.3이 A, B, C를 실행시킨 결과를 보이고 있다. SJF가 평균 반환 시간 기준으로 더 나은 성능을 보이는지 보인다. B, C, A 순서로 실행시킴으로써 SJF는 평균 반환 시간을 110초에서 50초 ($\frac{10+20+120}{3} = 50$)로 2배 이상 향상시켰다.



〈그림 10.3〉 SJF 간단한 예

모든 작업이 동시에 도착한다면 SJF가 **최적(optimal)**의 스케줄링 알고리즘임을 증명할 수 있다. 지금은 이론이나 오퍼레이션 리서치 수업이 아니라 시스템 수업을 듣고 있기 때문에 이에 대한 증명은 하지 않을 것이다.

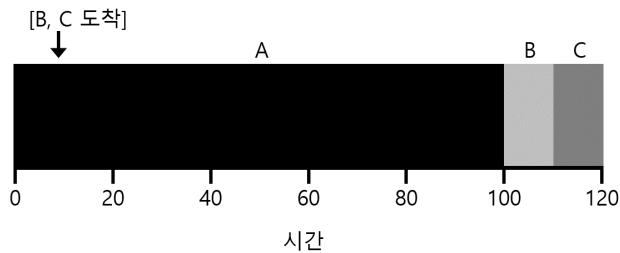
SJF라는 좋은 스케줄링 기법을 개발하였지만 가정이 아직 비현실적이다. 이제 가정을 완화해 보자. 특히, 가정 2를 완화하여 이제 모든 작업은 동시에 도착하는 것이 아니라 임의의 시간에 도착할 수 있다고 가정한다. 이러면 어떤 문제가 발생하는가?

(또 생각할 시간... 생각하고 있는 거지?... 자, 할 수 있어!)

자 문제가 무엇인지 설명하기 위하여 예를 들어 보자. 이번에는 A는 $t = 0$ 에 도착하고 100초의 실행 시간을 가지고 있고, 반면에 B와 C는 $t = 10$ 에 도착하고 각각 10초의 실행

별 문제 아니다, 걱정하지말자.

시간을 가진다고 가정하자. 순수한 SJF로 스케줄 했을 경우의 스케줄이 그림 10.4에 나와 있다.



〈그림 10.4〉 B와 C가 늦게 도착한 경우의 SJF 스케줄

그림에서 알 수 있듯이 B와 C가 A 바로 뒤에 도착한다고 하더라도 A가 끝날 때까지 기다릴 수밖에 없어서 이전의 convoy 문제가 다시 발생한다. 이 경우 평균 반환 시간은 $103.33 \left(\frac{100 + (110 - 10) + (120 - 10)}{3} \right)$ 이다. 스케줄러는 어떻게 스케줄링 해야 할까?

여담: 선점형 스케줄러

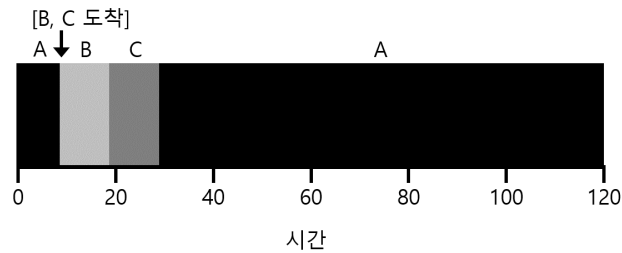
예전 일괄처리 시절에 많은 비선점(non-preemptive) 스케줄러가 개발되었다. 이 시스템은 각 작업이 종료될 때까지 계속 실행한다. 이론적으로 모든 현대 스케줄러는 선점형이고, 다른 프로세스를 실행시키기 위하여 필요하면 현재 프로세스의 실행을 중단한다. 이 사실은 스케줄러가 앞에서 배웠던 기법을 채용하고 있다는 것을 의미한다. 구체적으로 스케줄러는 문맥 교환을 수행할 수 있고, 실행 중인 프로세스를 잠시 중단시킬 수 있으며 다른 프로세스를 재개 또는 시작시킬 수 있다.

10.5 최소 잔여시간 우선

이 문제를 해결하기 위해서는 가정 3 (작업은 끝날 때까지 계속 실행된다)을 완화해야 한다. 이제 그 작업을 해 보자. 타이머 인터럽트와 문맥 교환을 고려하면 B와 C가 도착했을 때 스케줄러는 다른 일을 할 수 있다. 작업 A를 중지하고 지금 도착한 B나 C를 실행하기로 결정할 수도 있다. 물론, 선점된 A는 나중에 다시 실행된다. SJF는 비선점형 스케줄러이기 때문에 이와 같은 동작을 하지 못한다.

다행히도 이렇게 동작할 수 있는 스케줄러는 이미 존재한다. SJF에 선점 기능을 추가한 최단 잔여시간 우선(Shortest Time-to-Completion First, STCF) 또는 선점형 최단 작업 우선(PSJF)으로 알려진 스케줄러이다 [CK68]. 언제든지 새로운 작업이 시스템에 들어오면, 이 스케줄러는 남아 있는 작업과 새로운 작업의 잔여 실행 시간을 계산하고 그 중 가장 적은 잔여 실행 시간을 가진 작업을 스케줄한다. 우리의 예에서는

STCF는 A를 선점하고 B와 C를 끝날 때까지 실행시킨다. 두 작업이 모두 끝난 후에 A가 스케줄되어 남은 실행 시간 만큼 실행된다. 그림 10.5의 스케줄 예를 보라.



〈그림 10.5〉 STCF의 간단한 예

그 결과 평균 반환 시간이 단축되어 50초 ($\frac{((120-0)+(20-10)+(30-10))}{3}$)가 된다. 먼저와 같이, 새로운 가정 하에서 STCF가 최적의 스케줄러이다. 작업들이 동시에 도착할 경우, SFJ가 최적의 결과를 낸다는 것을 고려하면, STCF가 최적의 스케줄링이 되는 이유를 쉽게 알 수 있을것이다.

10.6 새로운 평가 기준: 응답 시간

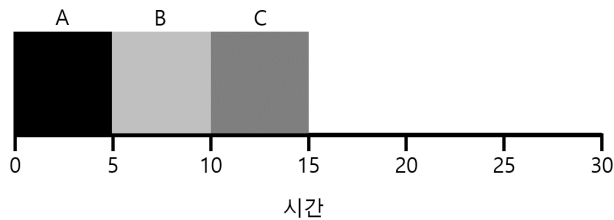
작업의 길이를 미리 알고 있고, 작업이 오직 CPU만 사용하며, 평가 기준이 반환 시간 하나라면, STCF는 매우 훌륭한 정책이다. 초기 일괄처리 컴퓨터 시스템에서는 이러한 스케줄링 알고리즘이 의미가 있었다. 그러나 시분할 컴퓨터의 등장이 모든 것을 바꾸었다. 이제 사용자는 터미널에서 작업하게 되어 시스템에게 상호작용을 원활히 하기 위한 성능을 요구하게 되었다. 응답 시간(response time)이라는 새로운 평가 기준이 태어나게 된다.

응답 시간은 작업이 도착할 때부터 처음 스케줄 될 때까지의 시간으로 정의된다. 다음과 같다.

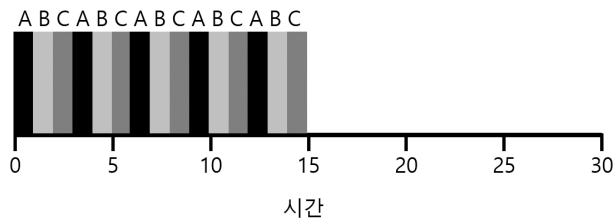
$$T_{response} = T_{firstrun} - T_{arrival} \quad (10.2)$$

예를 들어, 위와 같은 스케줄의 (A는 시간 0, B와 C는 시간 10에 도착) 경우, 각 작업의 응답 시간은 A는 0, B는 0, C는 10, 평균 3.33이 된다.

이미 생각하고 있는지 모르지만, STCF를 비롯하여 비슷한 류의 정책들은 응답 시간이 짧다고 할 수 없다. 예를 들어, 3개의 작업이 동시에 도착한 경우, 세 번째 작업은 딱 한 번 스케줄 되기 위해 먼저 실행된 두 작업이 완전히 끝날 때까지 기다린다. 반환 시간 기준으로는 매우 훌륭한 반면, 응답 시간과 상호작용 측면에서는 매우 나쁜 방법이다. 터미널 앞에 앉아 입력한 후, 단지 다른 작업이 당신의 작업보다 먼저 스케줄 되었다는 이유로 시스템의 응답이 올 때까지 10초 기다리는 것을 상상해 보자. 별로 즐거운 일은 아니다.



〈그림 10.6〉 다시 SJF 스케줄링 (응답 시간은 좋지 않음)



〈그림 10.7〉 라운드 로빈 (응답 시간이 좋음)

우리는 다른 문제를 해결해야 한다. 응답 시간에 민감한 스케줄러를 어떻게 만들 수 있는가?

10.7 라운드 로빈

응답 시간 문제를 해결하기 위하여 라운드 로빈(Round-Robin, RR) 스케줄링 [Kle64]이라 불리는 스케줄링 알고리즘을 도입한다. 기본 발상은 간단하다. RR은 작업이 끝날 때까지 기다리지 않는다. 대신 일정 시간 동안 실행한 후 실행 큐의 다음 작업으로 전환한다. 이때 작업이 실행되는 일정 시간을 **타임 슬라이스(time slice)** 또는 **스케줄링 쿼텀(scheduling quantum)**이라 부른다. 작업이 완료될 때까지 이런 식으로 계속 진행된다. 이러한 이유로 RR은 때때로 **타임 슬라이싱**이라고 불린다. 타임 슬라이스의 길이는 타이머 인터럽트 주기의 배수여야 한다. 타이머가 10 msec 마다 인터럽트를 발생시킨다면 타임 슬라이스는 10, 20 등 10 msec의 배수가 될 수 있다.

RR을 더 자세하게 이해하기 위해 하나의 예를 살펴보자. 3개의 작업 A, B, C가 시스템에 동시에 도착하고, 각각 5초간 실행된다고 가정한다. SJF 스케줄러는 다른 작업을 실행하기 전에 각 작업을 종료할 때까지 실행한다(그림 10.6). 대조적으로, 1초의 타임 슬라이스를 가지는 RR은 작업을 빠르게 순환하게 된다(그림 10.7). RR의 평균 응답 시간은 $\frac{(0+1+2)}{3} = 1$ 이고, SJF의 평균 응답 시간은 $\frac{(0+5+10)}{3} = 5$ 이다.

쉽게 알 수 있듯이, 타임 슬라이스의 길이는 RR에게 매우 중요하다. 타임 슬라이스가 짧을수록, 응답 시간 기준으로 RR의 성능은 더 좋아진다. 그러나 타임 슬라이스를 너무 짧게 지정하면 문제가 생긴다. 짧게 지정하면 문맥 교환 비용이 전체 성능에 큰 영향을 미치게 된다. 시스템 설계자는 시스템이 최적의 상태로 동작할 수 있도록 타임 슬라이스의

팁: 비용의 상쇄

일반적인 **상쇄(amortization)** 기술은 어떤 연산에 고정 비용이 존재하는 시스템에서 흔히 사용된다. 그 비용이 적은 횟수로 발생하게 하여, 즉 해당 연산을 적게 실행함으로써 시스템의 전체 비용이 감소된다. 예를 들어, 타임 슬라이스는 10 msec로 설정되고 문맥 교환 비용이 1 msec라면 대강 10%의 시간이 문맥 교환에 사용되고 따라서 낭비된다. 이 비용을 상쇄하고 싶은 경우, 타임 슬라이스를 예로 들어 100 msec로 늘릴 수 있다. 이러면 문맥 교환에 1% 미만의 시간이 소모되고 타임 슬라이싱의 비용이 상쇄된다.

길이를 결정해야 한다. 문맥 교환 비용을 **상쇄**할 수 있을 만큼 길어야 하지만 그렇다고 응답 시간이 너무 길어지면 안 된다.

문맥 교환 비용에는 레지스터를 저장/복원하는 작업만 있는 것이 아니다. CPU 캐시, TLB, 분기 예측 등을 비롯하여 다른 하드웨어에도 프로그램과 관련된 다양한 작업 정보들이 저장되어 있다. 작업이 전환되면 이 정보들은 모두 갱신되어야 한다. 갱신 작업이 매우 큰 성능 비용을 유발한다 [MB91].

적당한 길이의 응답 시간이 유일한 평가 기준인 경우 타임 슬라이스를 가진 RR은 매우 훌륭한 스케줄러이다. 반환 시간 기준으로서는 어떤가? 앞의 예를 다시 한 번 살펴보자. A, B, C 각 작업은 5초간 실행되고, 동시에 도착하며, RR은 1초의 타임 슬라이스를 가진다. A는 13, B는 14, C는 15에 종료하고 평균 14의 반환 시간을 보인다는 것을 알 수 있다. 매우 안 좋다!

반환 시간이 유일한 측정 기준인 경우 RR이 확실히 최악의 정책이라는 것은 놀라운 일이 아니다. 직관적으로, 충분히 이해 가능하다. RR은 각 작업을 잠깐 실행하고 다음 작업으로 넘어가고 하면서 가능한 한 각 작업을 늘리는 것이 목표다. 반환 시간은 작업 완료 시간만을 고려하기 때문에, RR은 거의 최악이며, 많은 경우 단순한 FIFO보다도 성능이 좋지 않게 된다.

일반적으로 RR과 같은 공정한 정책, 즉 작은 시간 단위로 모든 프로세스에게 CPU를 분배하는 정책은 반환 시간과 같은 평가 기준에서는 성능이 나쁘다. 이건 당연하다. 불공정하게 한다면 하나의 작업을 끝까지 실행하고 종료할 수 있지만 나머지 작업들에 대한 응답 시간은 포기해야 한다. 반대로 공정성을 더 중히 여긴다면 응답 시간은 줄어들이지만 반환 시간은 나빠지게 된다. 이러한 유형의 절충은 시스템에서는 흔히 있는 일이다. 당신은 케익을 먹으면서 동시에 케익을 보관할 수는 없다².

우리는 2종류의 스케줄러를 개발하였다. 첫째 유형은 (SJF, STCF) 반환 시간을 최적화하지만 응답 시간은 나쁘다. 두 번째 유형은 (RR) 응답 시간을 최적화하지만 반환 시간이 나쁘다. 아직 완화해야 하는 가정 두 개가 남아 있다. 가정 4 (작업은 입출력을 하지 않는다)와 가정 5 (각 작업의 실행 시간은 알려져 있다)이다. 이제 이 가정들을 정복해 보자.

2) 사람을 혼란시키는 속담. 원래는 “당신은 케익을 먹으면서 보존할 수 없다”여야 하기 때문이다. (분명하지 않은가, 아닌가?). 놀랍게도 이 속담에 관한 Wikipedia 페이지가 있다. 더욱 놀라운 것은 재미있는 읽을거리라는 것이다. http://en.wikipedia.org/wiki/You_can't_have_your_cake_and_eat_it

팁: 중첩은 높은 이용률을 가능하게 한다

가능하면 시스템의 활용도를 극대화하기 위해 연산을 중첩되게 실행한다. 중첩은 디스크 입출력을 할 때나 원격 컴퓨터에 메시지를 보낼 때 등 다양한 영역에서 유용하다. 어느 경우라도 이 작업들을 시작한 후에 다른 작업으로 전환하는 것은 좋은 생각이다. 시스템의 전반적인 사용률과 효율성이 향상된다.

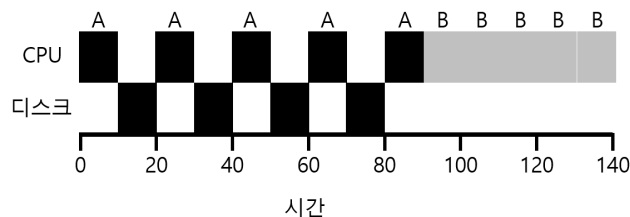
10.8 입출력 연산의 고려

우선 가정 4를 완화하자. 모든 프로그램은 입출력 작업을 수행한다. 아무런 입력도 받아들이지 않는 프로그램을 상상해 보자. 매번 같은 출력을 만들어 낼 것이다. 출력하지 않는 프로그램을 상상해 보자. 그건 속담에도 나올 법한 아무도 보는 사람이 없이 숲에서 잎이 떨어지는 것과 같다. 실행되었다는 사실이 아무 의미가 없다.

입출력 작업을 요청한 경우 스케줄러는 다음에 어떤 작업을 실행할지를 결정해야 한다. 현재 실행 중인 작업은 입출력이 완료될 때까지 CPU를 사용하지 않을 것이기 때문이다. 입출력 요청을 발생시킨 작업은 입출력 완료를 기다리며 대기 상태가 된다. 입출력이 하드 디스크 드라이브에 보내진 경우, 프로세스는 드라이브의 현재 입출력 워크로드에 따라 몇 초 또는 좀 더 긴 시간 동안 블록될 것이다. 스케줄러는 그 시간 동안 실행될 다른 작업을 스케줄 해야 한다.

마찬가지로 스케줄러는 입출력 완료 시에도 의사 결정을 해야 한다. 입출력이 완료되면 인터럽트가 발생하고 운영체제가 실행되어 입출력을 요청한 프로세스를 대기 상태에서 준비 상태로 다시 이동시킨다. 물론, 인터럽트가 발생했을 때 요청 프로세스를 즉시 실행시키기로 결정할 수도 있다. 운영체제는 각 작업을 어떻게 처리해야 하는가?

이 쟁점을 더 잘 이해하기 위해서 두 개의 작업 A와 B가 있다고 하자. 각 작업은 50 msec의 CPU 시간을 필요로 한다. 그러나 둘은 한 가지 큰 차이를 가진다. A는 10 msec 동안 실행된 후, 입출력 요청을 한다(여기서 입출력 시간은 10 msec라고 가정한다). 반면에 B는 입출력을 수행하지 않는다. 스케줄러는 A를 먼저 실행시키고 B를 다음에 실행시킨다(그림 10.8).

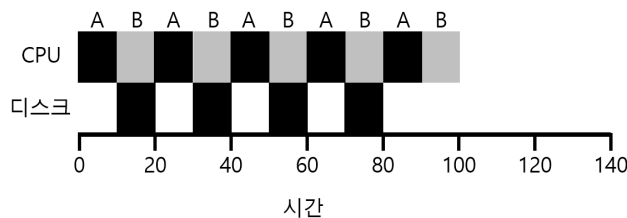


〈그림 10.8〉 자원의 비효율적인 활용

STCF 스케줄러를 구축하려고 한다고 가정하자. A가 5개의 10 msec 작업으로 분할되는 반면에 B는 하나의 50 msec의 CPU를 요청한다는 사실을 어떻게 활용할까?

분명히 입출력을 고려하지 않고 작업을 하나씩 실행시키는 것은 의미가 없다.

일반적인 접근 방식은 A의 각 10 msec 하위 작업을 독립적인 작업으로 다루는 것이다. 시스템이 시작할 때 10 msec 작업들과 50 msec를 스케줄하는 것이다. STCF의 경우 선택은 자명하다. 가장 짧은 작업을 선택하라. 이 경우에는 A가 된다. 그러면 A의 첫 번째 소 작업이 완료되면 B만 남게 되어 실행을 시작한다. A의 다음 작업이 제출되고 B를 선점하여 10 msec 동안 실행한다. 이렇게 하면 프로세스의 입출력이 끝나기를 기다리는 동안 CPU는 다른 프로세스에 의해 사용되어 연산의 중첩이 가능해진다. 시스템은 좀 더 잘 활용된다 (그림 10.9).



〈그림 10.9〉 중첩을 통해 효율적 자원 활용이 가능하다

입출력을 고려한 스케줄러 방법에 대해서 알아보았다. 각 CPU 버스트를 하나의 작업으로 간주함으로써 스케줄러는 대화형 프로세스가 더 자주, 즉 유리하게 실행되는 것을 보장한다. 이러한 대화형 작업이 입출력을 실행하는 동안 다른 CPU-집중 작업들이 실행되고 CPU의 이용률은 더 높아진다.

10.9 만병통치약은 없다 (No More Oracle)

입출력을 고려한 기본적인 접근 방식에 대해 논의하였으므로 마지막 가정에 대해 생각해 보자. 스케줄러가 각 작업의 실행 시간을 알고 있다는 가정이다. 전에 얘기했듯이, 이 가정은 아마 우리가 할 수 있는 최악의 가정이다. 사실 범용 운영체제에서(우리가 현재 고려 중인 것과 같은) 작업의 길이에 대해서 알 수 있는 길은 없다. 따라서 아무런 사전 지식 없이 SJF/STCF 처럼 행동하는 알고리즘을 구축할 수 있을까? 게다가, 응답 시간도 좋게 하기 위하여 RR 스케줄러의 경우에 보았던 아이디어를 어떻게 하면 포함시킬 수 있을까?

10.10 요약

우리는 스케줄링의 기본적인 개념과 두 가지 부류의 접근법을 살펴보았다. 첫 번째 부류는 남아 있는 작업 중 실행 시간이 제일 짧은 작업을 수행하고, 반환 시간을 최소화한다. 두 번째 부류는 모든 작업을 번갈아 실행시키고 응답 시간을 최소화한다. 안타깝게도 반환 시간과 응답 시간 중 하나를 최적화하면 나머지 하나는 좋지 않은 특성을 나타낸다. 이는 시스템에서 흔히 보이는 절충을 요구하는 상황이다. 전체적인 그림에 입출력을

어떻게 통합해야 하는지도 보았다. 그렇지만 미래를 예측할 수 없는 운영체제의 근본적인 문제는 해결할 수 없었다. 가까운 과거를 이용하여 미래를 예측하는 스케줄러를 구현하여 이 문제를 어떻게 해결하는지 곧 보게 될 것이다. 이 스케줄러는 **멀티 레벨 피드백 큐(multi-level feedback queue)**라고 불리며 다음 장의 주제이다.

참고 문헌

[Bla+79] **“The Convoy Phenomenon”**

M. Blasgen, J. Gray, M. Mitoma, and T. Price

ACM Operating Systems Review, 13:2, April 1979

아마도 운영체제뿐 아니라 데이터베이스에서 발생한 호위함 효과에 대한 첫 번째 참고 문헌.

[Cob54] **“Priority Assignment in Waiting Line Problems”**

A. Cobham

Journal of Operations Research, 2:70, pages 70-76, 1954

기계 수리 일정을 정하는 데 SJF 방식을 사용한 선구적인 논문.

[CK68] **“Computer Scheduling Methods and their Countermeasures”**

Edward G. Coffman and Leonard Kleinrock

AFIPS '68 (Spring), April 1968

다양한 기본 스케줄링 알고리즘에 대한 우수한 초기 소개 및 분석.

[Jai91] **“The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling”**

R. Jain

Interscience, New York, April 1991

컴퓨터 시스템 측정에 관한 표준 교과서. 당신의 라이브러리를 빛낼 훌륭한 참고 문헌.

[JV56] **“Machine Repair as a Priority Waiting-Line Problem”**

Thomas E. Phipps Jr. and W.R. Van Voorhis

Operations Research, 4:1, pages 76-86, February 1956

기계 수리 스케줄링에 사용된 Cobham의 SJF를 일반화한 후속 연구. 또한 이러한 환경에서 STCF 방식의 유용성을 생각한 논문. 구체적으로 “수리 작업에는 특정 유형이 존재한다 ... 많은 해체와 마루를 너트와 볼트로 뒤덮음을 동반하기 때문에 한 번 시작하면 중단되서는 안 된다. 다른 경우, 하나 이상의 짧은 작업들이 존재할 때 긴 작업을 계속하는 것은 권하고 싶지 않을 것이다 (p.81)”

[Kle64] **“Analysis of a Time-Shared Processor”**

Leonard Kleinrock

Naval Research Logistics Quarterly, 11:1, pages 59-73, March 1964

라운드 로빈 스케줄링 알고리즘에 대한 첫 번째 참고 문헌. 시분할 시스템을 스케줄링 할 때 라운드 로빈 스케줄링 알고리즘 사용에 대한 분석을 제시한 논문 중 하나.

[Lav78] **“The Manchester Mark I and Atlas: A Historical Perspective”**

S.H. Lavington

Communications of the ACM, 21:1, January 1978

초창기 컴퓨터 개발과 Atlas의 개척적인 노력에 관한 역사.

[MB91] **“The effect of context switches on cache performance”**

Jeffrey C. Mogul and Anita Borg

ASPLOS, 1991

문맥 교환이 캐시의 성능에 미치는 영향에 관한 좋은 연구. 프로세서는 초당 수십억 명령어를 실행하지만 문맥 교환은 msec 시간 단위로 발생하기 때문에 현재 시스템에서는 크게 문제가 되지 않는다.

[Orw45] **“Animal Farm”**

George Orwell

Secker and Warburg (London), 1945

권력과 그의 변질에 관한 슬프지만 우울한 우화책. 어떤 이는 제2차 세계대전 전 스탈린 시대의 소련에 대한 비판이라고 말하기도 한다. 우리는 그저 돼지에 대한 비판이라고 말한다.

숙제

`scheduler.py` 프로그램은 응답 시간, 반환 시간, 총 대기 시간의 관점에서 여러 스케줄러가 어떻게 동작하는지를 볼 수 있게 한다. 자세한 내용은 README를 참조하시오.

문제

1. 길이가 200인 세 개의 작업을 SJF와 FIFO 스케줄링 방식으로 실행할 경우 응답 시간과 반환 시간을 계산하시오.
2. 같은 조건이지만 작업의 길이가 각각 100, 200 및 300일 경우에 대해 계산하시오.
3. 2번과 같은 조건으로 타임 슬라이스가 1인 RR 스케줄러에 대해서도 계산하시오.
4. SJF와 FIFO가 같은 반환 시간을 보이는 워크로드의 유형은 무엇인가?
5. SJF가 RR과 같은 응답 시간을 보이기 위한 워크로드와 타임 퀀텀의 길이는 무엇인가?
6. 작업의 길이가 증가하면 SJF의 응답 시간은 어떻게 되는가? 변화의 추이를 보이기 위해서 시뮬레이터를 사용할 수 있는가?
7. 타임 퀀텀의 길이가 증가하면 RR의 응답 시간은 어떻게 되는가? N 개의 작업이 주어졌을 때, 최악의 응답 시간을 계산하는 식을 만들 수 있는가?

Multiprocessor Scheduling (Advanced)

This chapter will introduce the basics of **multiprocessor scheduling**. As this topic is relatively advanced, it may be best to cover it *after* you have studied the topic of concurrency in some detail (i.e., the second major “easy piece” of the book).

After years of existence only in the high-end of the computing spectrum, **multiprocessor** systems are increasingly commonplace, and have found their way into desktop machines, laptops, and even mobile devices. The rise of the **multicore** processor, in which multiple CPU cores are packed onto a single chip, is the source of this proliferation; these chips have become popular as computer architects have had a difficult time making a single CPU much faster without using (way) too much power. And thus we all now have a few CPUs available to us, which is a good thing, right?

Of course, there are many difficulties that arise with the arrival of more than a single CPU. A primary one is that a typical application (i.e., some C program you wrote) only uses a single CPU; adding more CPUs does not make that single application run faster. To remedy this problem, you’ll have to rewrite your application to run in **parallel**, perhaps using **threads** (as discussed in great detail in the second piece of this book). Multi-threaded applications can spread work across multiple CPUs and thus run faster when given more CPU resources.

ASIDE: ADVANCED CHAPTERS

Advanced chapters require material from a broad swath of the book to truly understand, while logically fitting into a section that is earlier than said set of prerequisite materials. For example, this chapter on multiprocessor scheduling makes much more sense if you’ve first read the middle piece on concurrency; however, it logically fits into the part of the book on virtualization (generally) and CPU scheduling (specifically). Thus, it is recommended such chapters be covered out of order; in this case, after the second piece of the book.

제10장 멀티프로세서 스케줄링 (고급)

멀티프로세서 스케줄링 / 영문 원문 1쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

이 장에서는 멀티프로세서 스케줄링의 기초를 소개한다. 비교적 고급 주제이므로, 병행성에 관해 어느 정도 자세히 공부한 뒤에, 즉 이 책의 두 번째 주요 '쉬운 부분'을 공부한 뒤에 다루는 것이 좋다.

여러 해 동안 고성능 컴퓨팅 분야에만 존재하던 멀티프로세서 시스템이 점점 흔해지고 있으며, 이제 데스크톱과 노트북은 물론 모바일 기기에도 사용되고 있다. 여러 CPU 코어를 하나의 칩에 집적한 멀티코어 프로세서의 등장이라는 이러한 확산을 이끌었다. 컴퓨터 설계자들이 전력을 지나치게 많이 사용하지 않고 단일 CPU의 속도를 크게 높이는 데 어려움을 겪으면서, 이런 칩이 널리 쓰이게 되었다. 따라서 이제 우리 모두 몇 개의 CPU를 사용할 수 있게 되었다. 좋은 일 아닌가?

물론 CPU가 하나보다 많아지면 여러 어려움이 생긴다. 가장 중요한 문제 중 하나는 일반적인 응용 프로그램, 예를 들어 여러분이 작성한 C 프로그램이 CPU를 하나만 사용한다는 것이다. CPU를 더 추가해도 그 응용 프로그램 하나가 더 빨리 실행되지는 않는다. 이 문제를 해결하려면 응용 프로그램을 병렬로 실행되도록 다시 작성해야 한다. 이를 위해 스레드를 사용할 수 있으며, 스레드는 이 책의 두 번째 부분에서 아주 자세히 다룬다. 멀티스레드 응용 프로그램은 작업을 여러 CPU에 분산할 수 있으므로 더 많은 CPU 자원을 제공받으면 더 빨리 실행될 수 있다.

보충 설명: 고급 장

고급 장을 제대로 이해하려면 이 책의 여러 부분에 걸친 내용이 필요하지만, 논리적인 구성상 그러한 선수 지식보다 앞선 부분에 들어간다. 예를 들어 멀티프로세서 스케줄링을 다루는 이 장은 책의 중간 부분인 병행성을 먼저 읽으면 훨씬 이해하기 쉽다. 그러나 논리적으로는 책의 가상화 부분, 더 구체적으로는 CPU 스케줄링 부분에 속한다. 따라서 이런 장은 책의 순서와 다르게 공부하는 것을 권장한다. 이 장의 경우 책의 두 번째 부분을 마친 뒤에 읽는 것이 좋다.

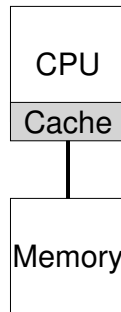


Figure 10.1: **Single CPU With Cache**

Beyond applications, a new problem that arises for the operating system is (not surprisingly!) that of **multiprocessor scheduling**. Thus far we've discussed a number of principles behind single-processor scheduling; how can we extend those ideas to work on multiple CPUs? What new problems must we overcome? And thus, our problem:

CRUX: HOW TO SCHEDULE JOBS ON MULTIPLE CPUS

How should the OS schedule jobs on multiple CPUs? What new problems arise? Do the same old techniques work, or are new ideas required?

10.1 Background: Multiprocessor Architecture

To understand the new issues surrounding multiprocessor scheduling, we have to understand a new and fundamental difference between single-CPU hardware and multi-CPU hardware. This difference centers around the use of hardware **caches** (e.g., Figure 10.1), and exactly how data is shared across multiple processors. We now discuss this issue further, at a high level. Details are available elsewhere [CSG99], in particular in an upper-level or perhaps graduate computer architecture course.

In a system with a single CPU, there are a hierarchy of **hardware caches** that in general help the processor run programs faster. Caches are small, fast memories that (in general) hold copies of *popular* data that is found in the main memory of the system. Main memory, in contrast, holds *all* of the data, but access to this larger memory is slower. By keeping frequently accessed data in a cache, the system can make the large, slow memory appear to be a fast one.

문제의 핵심과 멀티프로세서 구조

멀티프로세서 스케줄링 / 영문 원문 2쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

그림 10.1: 캐시가 있는 단일 CPU. 그림 표기: Memory = 메모리, CPU = CPU, Cache = 캐시. 그림은 인접한 영문 원본 페이지를 참조.

응용 프로그램 외에도 운영체제에는 새로운 문제가 생긴다. 예상했겠지만, 바로 멀티프로세서 스케줄링이다. 지금까지 단일 프로세서 스케줄링의 여러 원칙을 논의했다. 이 아이디어들을 여러 CPU에서도 작동하도록 어떻게 확장할 수 있을까? 어떤 새로운 문제를 극복해야 할까? 따라서 우리가 풀어야 할 문제는 다음과 같다.

핵심 질문: 여러 CPU에서 작업을 어떻게 스케줄링할 것인가

운영체제는 여러 CPU에서 작업을 어떻게 스케줄링해야 할까? 어떤 새로운 문제가 생길까? 기존 기법들을 그대로 사용할 수 있을까, 아니면 새로운 아이디어가 필요할까?

10.1 배경: 멀티프로세서 구조

멀티프로세서 스케줄링과 관련된 새로운 문제를 이해하려면 단일 CPU 하드웨어와 다중 CPU 하드웨어 사이의 새롭고 근본적인 차이를 이해해야 한다. 이 차이는 하드웨어 캐시의 사용, 예를 들어 그림 10.1과 같은 캐시의 사용, 그리고 여러 프로세서가 데이터를 공유하는 정확한 방식에 있다. 이제 이 문제를 개략적으로 더 살펴보자. 자세한 내용은 다른 자료 [CSG99]에서, 특히 학부 고학년 또는 대학원 컴퓨터 구조 수업에서 배울 수 있다.

단일 CPU 시스템에는 일반적으로 프로세서가 프로그램을 더 빨리 실행하도록 돕는 계층적인 하드웨어 캐시가 있다. 캐시는 시스템의 주 메모리에 있는 자주 사용되는 데이터의 복사본을 담은 작고 빠른 메모리이다. 이에 비해 주 메모리는 모든 데이터를 담지만, 용량이 큰 만큼 접근 속도가 느리다. 자주 접근하는 데이터를 캐시에 보관하면 시스템은 크고 느린 메모리가 빠른 메모리처럼 보이게 할 수 있다.

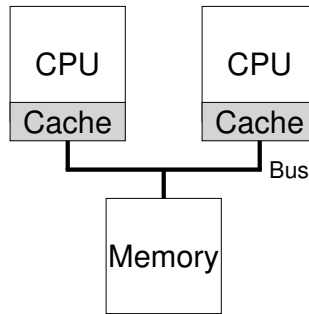


Figure 10.2: Two CPUs With Caches Sharing Memory

As an example, consider a program that issues an explicit load instruction to fetch a value from memory, and a simple system with only a single CPU; the CPU has a small cache (say 64 KB) and a large main memory. The first time a program issues this load, the data resides in main memory, and thus takes a long time to fetch (perhaps in the tens of nanoseconds, or even hundreds). The processor, anticipating that the data may be reused, puts a copy of the loaded data into the CPU cache. If the program later fetches this same data item again, the CPU first checks for it in the cache; if it finds it there, the data is fetched much more quickly (say, just a few nanoseconds), and thus the program runs faster.

Caches are thus based on the notion of **locality**, of which there are two kinds: **temporal locality** and **spatial locality**. The idea behind temporal locality is that when a piece of data is accessed, it is likely to be accessed again in the near future; imagine variables or even instructions themselves being accessed over and over again in a loop. The idea behind spatial locality is that if a program accesses a data item at address x , it is likely to access data items near x as well; here, think of a program streaming through an array, or instructions being executed one after the other. Because locality of these types exist in many programs, hardware systems can make good guesses about which data to put in a cache and thus work well.

Now for the tricky part: what happens when you have multiple processors in a single system, with a single shared main memory, as we see in Figure 10.2?

As it turns out, caching with multiple CPUs is much more complicated. Imagine, for example, that a program running on CPU 1 reads a data item (with value D) at address A ; because the data is not in the cache on CPU 1, the system fetches it from main memory, and gets the

캐시, 지역성, 공유 메모리

멀티프로세서 스케줄링 / 영문 원문 3쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

그림 10.2: 캐시를 갖추고 메모리를 공유하는 두 CPU. 그림 표기: Memory = 메모리, CPU = CPU, Cache = 캐시, Bus = 버스. 그림은 인접한 영문 원본 페이지를 참조.

예를 들어 메모리에서 값을 가져오기 위해 명시적인 load 명령을 실행하는 프로그램과, CPU가 하나뿐인 단순한 시스템을 생각해 보자. 이 CPU에는 작은 캐시, 예컨대 64KB 캐시와 큰 주 메모리가 있다. 프로그램이 이 load 명령을 처음 실행할 때 데이터는 주 메모리에 있으므로 이를 가져오는 데 오래 걸린다. 수십 나노초, 심하면 수백 나노초가 걸릴 수 있다. 프로세서는 이 데이터가 다시 사용될 것으로 예상하여 가져온 데이터의 복사본을 CPU 캐시에 넣는다. 프로그램이 나중에 같은 데이터를 다시 가져오면 CPU는 먼저 캐시에 그 데이터가 있는지 확인한다. 캐시에 있으면 데이터를 훨씬 빠르게, 예컨대 몇 나노초 만에 가져올 수 있으므로 프로그램이 더 빨리 실행된다.

따라서 캐시는 지역성이라는 개념에 바탕을 두며, 지역성에는 시간 지역성과 공간 지역성이라는 두 종류가 있다. 시간 지역성은 어떤 데이터에 접근하면 가까운 미래에 다시 접근할 가능성이 높다는 생각이다. 반복문 안에서 변수나 명령어 자체에 계속해서 접근하는 경우를 떠올려 보자. 공간 지역성은 프로그램이 주소 x의 데이터에 접근하면 x 근처의 데이터에도 접근할 가능성이 높다는 생각이다. 배열을 순서대로 읽는 프로그램이나 명령어를 하나씩 순서대로 실행하는 경우를 생각하면 된다. 많은 프로그램에서 이런 두 종류의 지역성이 나타나므로, 하드웨어 시스템은 어떤 데이터를 캐시에 넣을지 잘 추측할 수 있고 그 결과 효과적으로 작동한다.

이제 까다로운 부분이다. 그림 10.2처럼 하나의 시스템에 여러 프로세서가 있고 이들이 하나의 주 메모리를 공유하면 어떤 일이 일어날까?

여러 CPU가 있을 때의 캐싱은 훨씬 복잡하다. 예를 들어 CPU 1에서 실행되는 프로그램이 주소 A의 데이터, 즉 값 D를 읽는다고 하자. CPU 1의 캐시에 그 데이터가 없으므로 시스템은 주 메모리에서 이를 가져와...

value D . The program then modifies the value at address A , just updating its cache with the new value D' ; writing the data through all the way to main memory is slow, so the system will (usually) do that later. Then assume the OS decides to stop running the program and move it to CPU 2. The program then re-reads the value at address A ; there is no such data in CPU 2's cache, and thus the system fetches the value from main memory, and gets the old value D instead of the correct value D' . Oops!

This general problem is called the problem of **cache coherence**, and there is a vast research literature that describes many different subtleties involved with solving the problem [SHW11]. Here, we will skip all of the nuance and make some major points; take a computer architecture class (or three) to learn more.

The basic solution is provided by the hardware: by monitoring memory accesses, hardware can ensure that basically the “right thing” happens and that the view of a single shared memory is preserved. One way to do this on a bus-based system (as described above) is to use an old technique known as **bus snooping** [G83]; each cache pays attention to memory updates by observing the bus that connects them to main memory. When a CPU then sees an update for a data item it holds in its cache, it will notice the change and either **invalidate** its copy (i.e., remove it from its own cache) or **update** it (i.e., put the new value into its cache too). Write-back caches, as hinted at above, make this more complicated (because the write to main memory isn't visible until later), but you can imagine how the basic scheme might work.

10.2 Don't Forget Synchronization

Given that the caches do all of this work to provide coherence, do programs (or the OS itself) have to worry about anything when they access shared data? The answer, unfortunately, is yes, and is documented in great detail in the second piece of this book on the topic of concurrency. While we won't get into the details here, we'll sketch/review some of the basic ideas here (assuming you're familiar with concurrency).

When accessing (and in particular, updating) shared data items or structures across CPUs, mutual exclusion primitives (such as locks) should likely be used to guarantee correctness (other approaches, such as building **lock-free** data structures, are complex and only used on occasion; see the chapter on deadlock in the piece on concurrency for details). For example, assume we have a shared queue being accessed on multiple CPUs concurrently. Without locks, adding or removing elements from the queue concurrently will not work as expected, even with the underlying coherence protocols; one needs locks to atomically update the data structure to its new state.

To make this more concrete, imagine this code sequence, which is used to remove an element from a shared linked list, as we see in Figure 10.3. Imagine if threads on two CPUs enter this routine at the same time. If

캐시 일관성과 동기화

멀티프로세서 스케줄링 / 영문 원문 4쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

값 D를 얻는다. 그런 다음 프로그램은 주소 A의 값을 수정하는데, 캐시에만 새로운 값 D'를 반영한다. 데이터를 주 메모리까지 기록하는 일은 느리므로 시스템은 보통 나중에 수행한다. 이어서 운영체제가 프로그램 실행을 멈추고 이를 CPU 2로 옮기기로 했다고 하자. 프로그램이 주소 A의 값을 다시 읽으면 CPU 2의 캐시에는 해당 데이터가 없다. 따라서 시스템은 주 메모리에서 값을 가져오지만, 올바른 값 D' 대신 예전 값 D를 얻게 된다. 이런!

이 일반적인 문제를 캐시 일관성 문제라고 부른다. 이 문제를 해결할 때 고려해야 하는 여러 미묘한 사항을 설명하는 방대한 연구 문헌이 있다 [SHW11]. 여기서는 세부적인 사항을 생략하고 주요 내용만 짚는다. 더 배우고 싶다면 컴퓨터 구조 수업을 한 과목, 아니 세 과목쯤 들어 보자.

기본적인 해결책은 하드웨어가 제공한다. 하드웨어는 메모리 접근을 감시하여 기본적으로 '올바른 동작'이 일어나고 하나의 공유 메모리가 있다는 관점이 유지되도록 보장할 수 있다. 앞에서 설명한 버스 기반 시스템에서는 버스 스누핑이라는 오래된 기법을 사용할 수 있다 [G83]. 각 캐시는 자신을 주 메모리에 연결하는 버스를 관찰하여 메모리 갱신을 감시한다. 어떤 CPU가 자신의 캐시에 보관한 데이터의 갱신을 발견하면 그 변화를 알아차리고 자신의 복사본을 무효화하거나, 즉 자신의 캐시에서 제거하거나, 갱신한다. 갱신한다는 것은 새로운 값을 자신의 캐시에도 넣는다는 뜻이다. 앞서 암시했듯이 쓰기 지연 캐시는 주 메모리에 대한 쓰기가 나중에까지 보이지 않으므로 이 과정을 더 복잡하게 만든다. 그래도 기본적인 방식이 어떻게 작동할지는 짐작할 수 있을 것이다.

10.2 동기화를 잊지 말자

캐시가 일관성을 제공하기 위해 이렇게 많은 일을 한다면, 프로그램이나 운영체제 자체는 공유 데이터에 접근할 때 신경 쓸 일이 있을까? 안타깝지만 그렇다. 이 내용은 병행성을 다루는 이 책의 두 번째 부분에 아주 자세히 설명되어 있다. 여기서는 세부 사항에 들어가지 않고, 독자가 병행성을 알고 있다고 가정하여 몇 가지 기본 개념을 간단히 설명하거나 복습한다.

여러 CPU에서 공유 데이터 항목이나 자료구조에 접근할 때, 특히 이를 갱신할 때는 정확성을 보장하기 위해 락과 같은 상호 배제 기본 연산을 사용하는 것이 일반적이다. 락 없는 자료구조를 만드는 등의 다른 접근법도 있지만 복잡하며 제한적인 경우에만 사용된다. 자세한 내용은 병행성 부분의 교착 상태 장을 참조하자. 예를 들어 여러 CPU가 하나의 공유 큐에 동시에 접근한다고 하자. 락이 없으면 기본적인 일관성 프로토콜이 있더라도 큐에 원소를 동시에 추가하거나 제거하는 작업은 예상대로 동작하지 않는다. 자료구조를 새로운 상태로 원자적으로 갱신하려면 락이 필요하다.

더 구체적으로 살펴보기 위해 그림 10.3처럼 공유 연결 리스트에서 원소를 제거하는 코드를 생각해 보자. 두 CPU의 스레드가 동시에 이 루틴에 진입한다고 상상해 보자.

```

1  typedef struct __Node_t {
2      int          value;
3      struct __Node_t *next;
4  } Node_t;
5
6  int List_Pop() {
7      Node_t *tmp = head;          // remember old head
8      int value   = head->value;    // ... and its value
9      head       = head->next;      // advance to next
10     free(tmp);                    // free old head
11     return value;                 // return value @head
12 }

```

Figure 10.3: Simple List Delete Code

Thread 1 executes the first line, it will have the current value of `head` stored in its `tmp` variable; if Thread 2 then executes the first line as well, it also will have the same value of `head` stored in its own private `tmp` variable (`tmp` is allocated on the stack, and thus each thread will have its own private storage for it). Thus, instead of each thread removing an element from the head of the list, each thread will try to remove the same head element, leading to all sorts of problems (such as an attempted double free of the head element at Line 10, as well as potentially returning the same data value twice).

The solution, of course, is to make such routines correct via **locking**. In this case, allocating a simple mutex (e.g., `pthread_mutex_t m;`) and then adding a `lock(&m)` at the beginning of the routine and an `unlock(&m)` at the end will solve the problem, ensuring that the code will execute as desired. Unfortunately, as we will see, such an approach is not without problems, in particular with regards to performance. Specifically, as the number of CPUs grows, access to a synchronized shared data structure becomes quite slow.

10.3 One Final Issue: Cache Affinity

One final issue arises in building a multiprocessor cache scheduler, known as **cache affinity** [TTG95]. This notion is simple: a process, when run on a particular CPU, builds up a fair bit of state in the caches (and TLBs) of the CPU. The next time the process runs, it is often advantageous to run it on the same CPU, as it will run faster if some of its state is already present in the caches on that CPU. If, instead, one runs a process on a different CPU each time, the performance of the process will be worse, as it will have to reload the state each time it runs (note it will run correctly on a different CPU thanks to the cache coherence protocols of the hardware). Thus, a multiprocessor scheduler should consider cache affinity when making its scheduling decisions, perhaps preferring to keep a process on the same CPU if at all possible.

공유 리스트와 캐시 친화성

멀티프로세서 스케줄링 / 영문 원문 5쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

```

1 typedef struct __Node_t {
2     int value;
3     struct __Node_t *next;
4 } Node_t;
5
6 int List_Pop() {
7     Node_t *tmp = head;    // 기존 head를 기억한다
8     int value = head->value; // 그 값도 기억한다
9     head = head->next;     // 다음 원소로 이동한다
10    free(tmp);             // 기존 head를 해제한다
11    return value;          // head에 있던 값을 반환한다
12 }
```

그림 10.3: 간단한 리스트 삭제 코드.

스레드 1이 첫 번째 문장을 실행하면 현재 head 값이 자신의 tmp 변수에 저장된다. 이어서 스레드 2도 첫 번째 문장을 실행하면 자신의 전용 tmp 변수에 같은 head 값이 저장된다. tmp는 스택에 할당되므로 각 스레드는 이를 위한 별도의 저장 공간을 갖는다. 따라서 각 스레드가 리스트의 머리에서 원소를 하나씩 제거하는 대신, 두 스레드 모두 같은 머리 원소를 제거하려고 한다. 그 결과 10행에서 머리 원소를 두 번 해제하려 하거나 같은 데이터 값을 두 번 반환할 수 있는 등 여러 문제가 생긴다.

물론 해결책은 락을 사용하여 이러한 루틴이 올바르게 동작하도록 만드는 것이다. 여기서는 단순한 뮤텍스, 예를 들어 pthread_mutex_t m;을 할당하고 루틴의 시작 부분에 lock(&m)을, 끝부분에 unlock(&m)을 추가하면 문제가 해결되어 코드가 의도한 대로 실행된다. 안타깝게도 앞으로 보겠지만 이 접근법에도 문제가 있으며, 특히 성능과 관련된 문제가 있다. 구체적으로 CPU 수가 늘어나면 동기화된 공유 자료구조에 접근하는 속도가 상당히 느려진다.

10.3 마지막 문제: 캐시 친화성

멀티프로세서용 캐시를 고려하는 스케줄러를 만들 때는 캐시 친화성이라는 마지막 문제가 생긴다 [TTG95]. 개념은 간단하다. 어떤 프로세스가 특정 CPU에서 실행되면 그 CPU의 캐시와 TLB에 상당한 상태가 쌓인다. 다음에 이 프로세스가 실행될 때도 같은 CPU에서 실행하는 것이 유리한 경우가 많다. 그 CPU의 캐시에 상태의 일부가 이미 남아 있으면 더 빨리 실행되기 때문이다. 반대로 매번 다른 CPU에서 프로세스를 실행하면 실행할 때마다 상태를 다시 불러와야 하므로 성능이 떨어진다. 하드웨어의 캐시 일관성 프로토콜 덕분에 다른 CPU에서도 올바르게 실행되기는 한다. 따라서 멀티프로세서 스케줄러는 스케줄링을 결정할 때 캐시 친화성을 고려해야 하며, 가능하다면 프로세스를 같은 CPU에 유지하는 쪽을 선호할 수 있다.

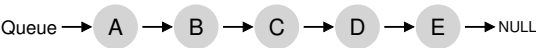
10.4 Single-Queue Scheduling

With this background in place, we now discuss how to build a scheduler for a multiprocessor system. The most basic approach is to simply reuse the basic framework for single processor scheduling, by putting all jobs that need to be scheduled into a single queue; we call this **single-queue multiprocessor scheduling** or **SQMS** for short. This approach has the advantage of simplicity; it does not require much work to take an existing policy that picks the best job to run next and adapt it to work on more than one CPU (where it might pick the best two jobs to run, if there are two CPUs, for example).

However, SQMS has obvious shortcomings. The first problem is a lack of **scalability**. To ensure the scheduler works correctly on multiple CPUs, the developers will have inserted some form of **locking** into the code, as described above. Locks ensure that when SQMS code accesses the single queue (say, to find the next job to run), the proper outcome arises.

Locks, unfortunately, can greatly reduce performance, particularly as the number of CPUs in the systems grows [A90]. As contention for such a single lock increases, the system spends more and more time in lock overhead and less time doing the work the system should be doing (note: it would be great to include a real measurement of this in here someday).

The second main problem with SQMS is cache affinity. For example, let us assume we have five jobs to run (*A, B, C, D, E*) and four processors. Our scheduling queue thus looks like this:



Over time, assuming each job runs for a time slice and then another job is chosen, here is a possible job schedule across CPUs:

CPU 0						... (repeat) ...
CPU 1						... (repeat) ...
CPU 2						... (repeat) ...
CPU 3						... (repeat) ...

Because each CPU simply picks the next job to run from the globally-shared queue, each job ends up bouncing around from CPU to CPU, thus doing exactly the opposite of what would make sense from the standpoint of cache affinity.

To handle this problem, most SQMS schedulers include some kind of affinity mechanism to try to make it more likely that process will continue

10.4 단일 큐 스케줄링

멀티프로세서 스케줄링 / 영문 원문 6쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

이러한 배경을 바탕으로 이제 멀티프로세서 시스템용 스케줄러를 어떻게 만들지 살펴보자. 가장 기본적인 접근법은 스케줄링해야 할 모든 작업을 하나의 큐에 넣어 단일 프로세서 스케줄링의 기본 틀을 그대로 재사용하는 것이다. 이를 단일 큐 멀티프로세서 스케줄링, 줄여서 SQMS라고 한다. 이 접근법의 장점은 단순함이다. 다음에 실행할 가장 적절한 작업을 선택하는 기존 정책을 가져와 여러 CPU에서 작동하도록 바꾸는 데는 많은 작업이 필요하지 않다. 예를 들어 CPU가 두 개라면 실행할 가장 적절한 작업 두 개를 고르면 된다.

하지만 SQMS에는 뚜렷한 단점이 있다. 첫 번째 문제는 확장성 부족이다. 스케줄러가 여러 CPU에서 올바르게 작동하도록 개발자는 앞서 설명한 것처럼 코드에 어떤 형태로든 락을 넣어야 한다. 락은 SQMS 코드가 다음에 실행할 작업을 찾는 등의 목적으로 단일 큐에 접근할 때 올바른 결과를 보장한다.

안타깝게도 락은 성능을 크게 떨어뜨릴 수 있으며, 특히 시스템의 CPU 수가 늘어날수록 그렇다 [A90]. 하나의 락을 둘러싼 경합이 증가하면 시스템은 락에 드는 부가 비용에 점점 더 많은 시간을 쓰고, 실제로 해야 할 작업에는 더 적은 시간을 쓰게 된다. 참고로, 언젠가 이 부분에 실제 측정 결과를 넣으면 좋겠다.

SQMS의 두 번째 주요 문제는 캐시 친화성이다. 예를 들어 실행할 작업이 다섯 개, A, B, C, D, E이고 프로세서는 네 개라고 하자. 스케줄링 큐는 다음과 같다.

큐 $\rightarrow A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow \text{NULL}$

각 작업이 한 타임 슬라이스 동안 실행된 다음 다른 작업이 선택된다고 가정하면, 시간이 흐름에 따라 CPU별로 다음과 같은 스케줄이 만들어질 수 있다.

CPU	시간 순서에 따른 작업
CPU 0	$A \rightarrow E \rightarrow D \rightarrow C \rightarrow B \rightarrow \dots$ (반복)
CPU 1	$B \rightarrow A \rightarrow E \rightarrow D \rightarrow C \rightarrow \dots$ (반복)
CPU 2	$C \rightarrow B \rightarrow A \rightarrow E \rightarrow D \rightarrow \dots$ (반복)
CPU 3	$D \rightarrow C \rightarrow B \rightarrow A \rightarrow E \rightarrow \dots$ (반복)

각 CPU가 전역 공유 큐에서 단순히 다음 작업을 선택하므로 각 작업은 CPU 사이를 계속 옮겨 다니게 된다. 이는 캐시 친화성의 관점에서 바람직한 동작과 정반대이다.

이 문제를 해결하기 위해 대부분의 SQMS 스케줄러는 가능하면 프로세스가 같은 CPU에서 계속 실행될 가능성을 높이려는 어떤 형태의 친화성 기법을 포함한다.

to run on the same CPU if possible. Specifically, one might provide affinity for some jobs, but move others around to balance load. For example, imagine the same five jobs scheduled as follows:

CPU 0	A	E	A	A	A	... (repeat) ...
CPU 1	B	B	E	B	B	... (repeat) ...
CPU 2	C	C	C	E	C	... (repeat) ...
CPU 3	D	D	D	D	E	... (repeat) ...

In this arrangement, jobs *A* through *D* are not moved across processors, with only job *E* **migrating** from CPU to CPU, thus preserving affinity for most. You could then decide to migrate a different job the next time through, thus achieving some kind of affinity fairness as well. Implementing such a scheme, however, can be complex.

Thus, we can see the SQMS approach has its strengths and weaknesses. It is straightforward to implement given an existing single-CPU scheduler, which by definition has only a single queue. However, it does not scale well (due to synchronization overheads), and it does not readily preserve cache affinity.

10.5 Multi-Queue Scheduling

Because of the problems caused in single-queue schedulers, some systems opt for multiple queues, e.g., one per CPU. We call this approach **multi-queue multiprocessor scheduling** (or **MQMS**).

In MQMS, our basic scheduling framework consists of multiple scheduling queues. Each queue will likely follow a particular scheduling discipline, such as round robin, though of course any algorithm can be used. When a job enters the system, it is placed on exactly one scheduling queue, according to some heuristic (e.g., random, or picking one with fewer jobs than others). Then it is scheduled essentially independently, thus avoiding the problems of information sharing and synchronization found in the single-queue approach.

For example, assume we have a system where there are just two CPUs (labeled CPU 0 and CPU 1), and some number of jobs enter the system: *A*, *B*, *C*, and *D* for example. Given that each CPU has a scheduling queue now, the OS has to decide into which queue to place each job. It might do something like this:



친화성 유지와 다중 큐 스케줄링

멀티프로세서 스케줄링 / 영문 원문 7쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

구체적으로 일부 작업에는 친화성을 제공하면서, 부하의 균형을 맞추기 위해 다른 작업들은 이동시킬 수 있다. 예를 들어 같은 다섯 작업을 다음처럼 스케줄링한다고 하자.

CPU	시간 순서에 따른 작업
CPU 0	A → E → A → A → A → ... (반복)
CPU 1	B → B → E → B → B → ... (반복)
CPU 2	C → C → C → E → C → ... (반복)
CPU 3	D → D → D → D → E → ... (반복)

이 배치에서는 A부터 D까지의 작업은 프로세서 사이를 이동하지 않고 E만 CPU 사이를 이동하므로 대부분 작업의 친화성이 유지된다. 다음번에는 다른 작업을 이동시키기로 하여 친화성 측면에서도 어느 정도 공정성을 달성할 수 있다. 그러나 이러한 방식을 구현하는 것은 복잡할 수 있다.

따라서 SQMS 접근법에는 장단점이 있다. 정의상 큐를 하나만 갖는 기존 단일 CPU 스케줄러가 있다면 구현하기 쉽다. 그러나 동기화에 따른 부가 비용 때문에 확장성이 좋지 않고, 캐시 친화성을 쉽게 유지하지도 못한다.

10.5 다중 큐 스케줄링

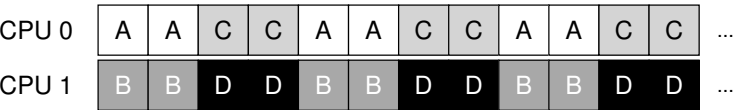
단일 큐 스케줄러에서 발생하는 문제 때문에 어떤 시스템은 여러 큐를 선택한다. 예를 들어 CPU마다 큐를 하나씩 둔다. 이를 다중 큐 멀티프로세서 스케줄링, 줄여서 MQMS라고 한다.

MQMS의 기본 스케줄링 틀은 여러 스케줄링 큐로 구성된다. 각 큐는 보통 라운드 로빈 같은 특정 스케줄링 방식을 따르지만, 물론 어떤 알고리즘이든 사용할 수 있다. 작업이 시스템에 들어오면 임의로 고르거나 다른 큐보다 작업 수가 적은 큐를 고르는 등의 경험적 방법에 따라 정확히 하나의 스케줄링 큐에 배치된다. 이후에는 본질적으로 독립적으로 스케줄링되므로 단일 큐 접근법에서 발생하는 정보 공유와 동기화 문제를 피할 수 있다.

예를 들어 CPU 0과 CPU 1이라는 두 CPU만 있는 시스템에 A, B, C, D 같은 몇 개의 작업이 들어온다고 하자. 이제 각 CPU가 스케줄링 큐를 가지므로 운영체제는 각 작업을 어느 큐에 넣을지 결정해야 한다. 다음과 같이 할 수 있다.

큐	작업
Q0	A → C
Q1	B → D

Depending on the queue scheduling policy, each CPU now has two jobs to choose from when deciding what should run. For example, with **round robin**, the system might produce a schedule that looks like this:

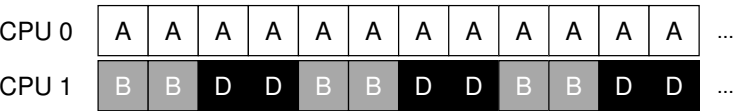


MQMS has a distinct advantage over SQMS in that it should be inherently more scalable. As the number of CPUs grows, so too does the number of queues, and thus lock and cache contention should not become a central problem. In addition, MQMS intrinsically provides cache affinity; jobs stay on the same CPU and thus reap the advantage of reusing cached contents therein.

But, if you’ve been paying attention, you might see that we have a new problem, which is fundamental in the multi-queue based approach: **load imbalance**. Let’s assume we have the same set up as above (four jobs, two CPUs), but then one of the jobs (say *C*) finishes. We now have the following scheduling queues:



If we then run our round-robin policy on each queue of the system, we will see this resulting schedule:



As you can see from this diagram, *A* gets twice as much CPU as *B* and *D*, which is not the desired outcome. Even worse, let’s imagine that both *A* and *C* finish, leaving just jobs *B* and *D* in the system. The two scheduling queues, and resulting timeline, will look like this:



How terrible – CPU 0 is idle! (*insert dramatic and sinister music here*) And thus our CPU usage timeline looks quite sad.

다중 큐의 장점과 부하 불균형

멀티프로세서 스케줄링 / 영문 원문 8쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

이제 각 CPU는 큐의 스케줄링 정책에 따라 두 작업 중 하나를 선택하여 실행한다. 예를 들어 라운드 로빈을 사용하면 시스템은 다음과 같은 스케줄을 만들 수 있다.

CPU	시간 순서에 따른 작업
CPU 0	A A C C A A C C A A C C ...
CPU 1	B B D D B B D D B B D D ...

MQMS는 본질적으로 확장성이 더 높다는 점에서 SQMS보다 뚜렷한 장점이 있다. CPU 수가 늘어나면 큐 수도 늘어나므로 락과 캐시 경합이 핵심적인 문제가 되지 않을 것이다. 또한 MQMS는 본래 캐시 친화성을 제공한 다. 작업이 같은 CPU에 머무르므로 그 캐시에 있는 내용을 재사용하는 이점을 얻는다.

하지만 주의 깊게 읽었다면 다중 큐 접근법에 근본적인 새로운 문제가 생긴다는 사실을 눈치챘을 것이다. 바로 부하 불균형이다. 위와 똑같이 작업 네 개와 CPU 두 개가 있지만, 그중 하나의 작업, 예를 들어 C가 끝났다고 하자. 그러면 스케줄링 큐는 다음과 같다.

큐	작업
Q0	A
Q1	B → D

시스템의 각 큐에 라운드 로빈 정책을 적용하면 다음과 같은 스케줄이 나온다.

CPU	시간 순서에 따른 작업
CPU 0	A A A A A A A A A A A ...
CPU 1	B B D D B B D D B B D D ...

이 그림에서 보듯 A는 B나 D보다 CPU 시간을 두 배나 많이 받는다. 이는 원하는 결과가 아니다. 더 나쁜 경우로, A와 C가 모두 끝나고 B와 D만 시스템에 남았다고 하자. 두 스케줄링 큐와 그에 따른 시간표는 다음과 같아 진다.

큐	작업
Q0	비어 있음
Q1	B → D

CPU	시간 순서에 따른 작업
CPU 0	유휴 상태
CPU 1	B B D D B B D D B B D D ...

끔찍하다. CPU 0이 놀고 있다! 여기에 극적이고 불길한 음악을 깔아 보자. 이렇게 CPU 사용 시간표는 매우 안타까운 모습이 된다.

So what should a poor multi-queue multiprocessor scheduler do? How can we overcome the insidious problem of load imbalance and defeat the evil forces of ... the Decepticons¹? How do we stop asking questions that are hardly relevant to this otherwise wonderful book?

CRUX: HOW TO DEAL WITH LOAD IMBALANCE

How should a multi-queue multiprocessor scheduler handle load imbalance, so as to better achieve its desired scheduling goals?

The obvious answer to this query is to move jobs around, a technique which we (once again) refer to as **migration**. By migrating a job from one CPU to another, true load balance can be achieved.

Let's look at a couple of examples to add some clarity. Once again, we have a situation where one CPU is idle and the other has some jobs.



In this case, the desired migration is easy to understand: the OS should simply move one of *B* or *D* to CPU 0. The result of this single job migration is evenly balanced load and everyone is happy.

A more tricky case arises in our earlier example, where *A* was left alone on CPU 0 and *B* and *D* were alternating on CPU 1:



In this case, a single migration does not solve the problem. What would you do in this case? The answer, alas, is continuous migration of one or more jobs. One possible solution is to keep switching jobs, as we see in the following timeline. In the figure, first *A* is alone on CPU 0, and *B* and *D* alternate on CPU 1. After a few time slices, *B* is moved to compete with *A* on CPU 0, while *D* enjoys a few time slices alone on CPU 1. And thus load is balanced:

CPU 0	A	A	A	A	B	A	B	A	B	B	B	B	...
CPU 1	B	D	B	D	D	D	D	D	A	D	A	D	...

Of course, many other possible migration patterns exist. But now for the tricky part: how should the system decide to enact such a migration?

¹Little known fact is that the home planet of Cybertron was destroyed by bad CPU scheduling decisions. And now let that be the first and last reference to Transformers in this book, for which we sincerely apologize.

부하 불균형과 작업 이동

멀티프로세서 스케줄링 / 영문 원문 9쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

그렇다면 불쌍한 다중 큐 멀티프로세서 스케줄러는 어떻게 해야 할까? 어떻게 하면 교묘한 부하 불균형 문제를 극복하고 사악한 세력인... 디셉티콘을 물리칠 수 있을까? [각주 1] 다른 부분은 이렇게 훌륭한 이 책에서, 별로 관련도 없는 질문을 그만하려면 어떻게 해야 할까?

핵심 질문: 부하 불균형에 어떻게 대처할 것인가

다중 큐 멀티프로세서 스케줄러는 원하는 스케줄링 목표를 더 잘 달성하기 위해 부하 불균형을 어떻게 처리해야 할까?

이 질문의 분명한 답은 작업을 이리저리 옮기는 것이다. 우리는 이 기법을 다시 한 번 이주, 즉 마이그레이션이라고 부른다. 작업을 한 CPU에서 다른 CPU로 옮기면 실질적인 부하 균형을 이룰 수 있다.

몇 가지 예로 더 분명히 살펴보자. 다시 한 CPU는 유휴 상태이고 다른 CPU에는 몇 개의 작업이 있는 상황이다.

큐	작업
Q0	비어 있음
Q1	B → D

이 경우 바람직한 이동은 이해하기 쉽다. 운영체제가 B나 D 중 하나를 CPU 0으로 옮기기만 하면 된다. 이 작업 하나를 옮긴 결과 부하가 고르게 균형을 이루고 모두가 행복해진다.

앞선 예처럼 CPU 0에는 A만 남고 CPU 1에서는 B와 D가 번갈아 실행되는 경우에는 더 까다로운 문제가 생긴다.

큐	작업
Q0	A
Q1	B → D

이 경우 한 번의 이동만으로는 문제가 해결되지 않는다. 여러분이라면 어떻게 하겠는가? 안타깝지만 답은 하나 이상의 작업을 계속 이동시키는 것이다. 가능한 해결책 하나는 다음 시간표처럼 작업의 위치를 계속 바꾸는 것이다. 그림에서는 처음에 A가 CPU 0을 혼자 사용하고, B와 D는 CPU 1에서 번갈아 실행된다. 몇 타임 슬라이스가 지난 뒤 B를 CPU 0으로 옮겨 A와 경쟁하게 하고, D는 CPU 1에서 몇 타임 슬라이스를 혼자 누리게 한다. 이렇게 부하의 균형이 맞춰진다.

CPU	시간 순서에 따른 작업
CPU 0	A A A B A B A B B B B ...
CPU 1	B D B D D D D A D A D ...

물론 이 외에도 가능한 이동 패턴은 많다. 하지만 이제 어려운 부분이다. 시스템은 이러한 이동을 언제 수행할지 어떻게 결정해야 할까?

각주 1: 잘 알려지지 않은 사실이지만, 고향 행성 사이버트론은 잘못된 CPU 스케줄링 결정 때문에 파괴되었다. 이 책에서 트랜스포머를 언급하는 것은 이것이 처음이자 마지막이 되도록 하자. 진심으로 사과드린다.

One basic approach is to use a technique known as **work stealing** [FLR98]. With a work-stealing approach, a (source) queue that is low on jobs will occasionally peek at another (target) queue, to see how full it is. If the target queue is (notably) more full than the source queue, the source will “steal” one or more jobs from the target to help balance load.

Of course, there is a natural tension in such an approach. If you look around at other queues too often, you will suffer from high overhead and have trouble scaling, which was the entire purpose of implementing the multiple queue scheduling in the first place! If, on the other hand, you don’t look at other queues very often, you are in danger of suffering from severe load imbalances. Finding the right threshold remains, as is common in system policy design, a black art.

10.6 Linux Multiprocessor Schedulers

Interestingly, in the Linux community, no common solution has emerged to building a multiprocessor scheduler. Over time, three different schedulers arose: the O(1) scheduler, the Completely Fair Scheduler (CFS), and the BF Scheduler (BFS)². See Meehan’s dissertation for an excellent overview of the strengths and weaknesses of said schedulers [M11]; here we just summarize a few of the basics.

Both O(1) and CFS use multiple queues, whereas BFS uses a single queue, showing that both approaches can be successful. Of course, there are many other details which separate these schedulers. For example, the O(1) scheduler is a priority-based scheduler (similar to the MLFQ discussed before), changing a process’s priority over time and then scheduling those with highest priority in order to meet various scheduling objectives; interactivity is a particular focus. CFS, in contrast, is a deterministic proportional-share approach (more like Stride scheduling, as discussed earlier). BFS, the only single-queue approach among the three, is also proportional-share, but based on a more complicated scheme known as Earliest Eligible Virtual Deadline First (EEVDF) [SA96]. Read more about these modern algorithms on your own; you should be able to understand how they work now!

10.7 Summary

We have seen various approaches to multiprocessor scheduling. The single-queue approach (SQMS) is rather straightforward to build and balances load well but inherently has difficulty with scaling to many processors and cache affinity. The multiple-queue approach (MQMS) scales better and handles cache affinity well, but has trouble with load imbalance and is more complicated. Whichever approach you take, there is no simple answer: building a general purpose scheduler remains a daunting task, as small code changes can lead to large behavioral differences. Only undertake such an exercise if you know exactly what you are doing, or, at least, are getting paid a large amount of money to do so.

²Look up what BF stands for on your own; be forewarned, it is not for the faint of heart.

작업 훔치기, Linux 스케줄러, 정리

멀티프로세서 스케줄링 / 영문 원문 10쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

기본적인 접근법 하나는 작업 훔치기라는 기법을 사용하는 것이다 [FLR98]. 작업 훔치기에서는 작업이 부족한 출발 큐가 가끔 다른 대상 큐를 들여다보며 얼마나 차 있는지 확인한다. 대상 큐가 출발 큐보다 눈에 띄게 더 많이 차 있으면 출발 큐는 대상 큐에서 하나 이상의 작업을 '훔쳐' 부하 균형을 맞춘다.

물론 이 접근법에는 자연스러운 상충 관계가 있다. 다른 큐를 너무 자주 살펴보면 부가 비용이 커지고 확장성이 나빠진다. 애초에 다중 큐 스케줄링을 구현한 목적이 바로 확장성이었는데 말이다! 반대로 다른 큐를 충분히 자주 확인하지 않으면 심각한 부하 불균형을 겪을 위험이 있다. 적절한 임계값을 찾는 일은 시스템 정책 설계에서 흔히 그렇듯 여전히 경험과 요령이 필요한 기술이다.

10.6 Linux 멀티프로세서 스케줄러

흥미롭게도 Linux 커뮤니티에서는 멀티프로세서 스케줄러를 만드는 데 공통된 해결책 하나가 자리 잡지 못했다. 시간이 흐르면서 $O(1)$ 스케줄러, 완전 공정 스케줄러(CFS), BF 스케줄러(BFS)라는 세 가지 서로 다른 스케줄러가 등장했다. [각주 2] 이 스케줄러들의 장단점을 훌륭하게 정리한 자료는 Meehan의 학위 논문 [M11]을 참조하자. 여기서는 몇 가지 기본 사항만 요약한다.

$O(1)$ 과 CFS는 모두 다중 큐를 사용하고 BFS는 단일 큐를 사용한다. 이는 두 접근법 모두 성공할 수 있음을 보여 준다. 물론 이 스케줄러들은 다른 세부 사항에서도 여러 차이가 있다. 예를 들어 $O(1)$ 스케줄러는 앞서 다룬 MLFQ와 비슷한 우선순위 기반 스케줄러이다. 시간에 따라 프로세스의 우선순위를 바꾸고 가장 높은 우선순위를 가진 프로세스부터 스케줄링하여 여러 스케줄링 목표를 달성하려 하며, 특히 상호작용성을 중시한다. 반면 CFS는 앞서 설명한 스트라이드 스케줄링에 더 가까운 결정론적 비례 배분 방식이다. 세 가지 중 유일한 단일 큐 방식인 BFS도 비례 배분 방식이지만, Earliest Eligible Virtual Deadline First(EEVDF)라는 더 복잡한 방식에 기반한다 [SA96]. 이 현대적인 알고리즘들은 스스로 더 읽어 보자. 이제 어떻게 작동하는지 이해할 수 있을 것이다!

10.7 요약

멀티프로세서 스케줄링의 여러 접근법을 살펴보았다. 단일 큐 접근법(SQMS)은 비교적 만들기 쉽고 부하 균형도 잘 맞추지만, 본질적으로 많은 프로세서로 확장하거나 캐시 친화성을 유지하는 데 어려움이 있다. 다중 큐 접근법(MQMS)은 확장성이 더 좋고 캐시 친화성도 잘 처리하지만, 부하 불균형 문제가 있으며 더 복잡하다. 어떤 접근법을 택하든 간단한 정답은 없다. 작은 코드 변경이 동작의 큰 차이로 이어질 수 있으므로 범용 스케줄러를 만드는 일은 여전히 만만치 않다. 자신이 무엇을 하는지 정확히 알거나, 적어도 그 일을 하는 대가로 많은 돈을 받는 경우에만 이런 작업에 도전하자.

각주 2: BF가 무엇의 약자인지는 직접 찾아보자. 미리 경고하지만, 마음이 약한 사람에게는 권하지 않는다.

References

- [A90] “The Performance of Spin Lock Alternatives for Shared-Memory Multiprocessors” by Thomas E. Anderson. IEEE TPDS Volume 1:1, January 1990. *A classic paper on how different locking alternatives do and don't scale. By Tom Anderson, very well known researcher in both systems and networking. And author of a very fine OS textbook, we must say.*
- [B+10] “An Analysis of Linux Scalability to Many Cores Abstract” by Silas Boyd-Wickizer, Austin T. Clements, Yandong Mao, Aleksey Pesterev, M. Frans Kaashoek, Robert Morris, Nickolai Zeldovich. OSDI '10, Vancouver, Canada, October 2010. *A terrific modern paper on the difficulties of scaling Linux to many cores.*
- [CSG99] “Parallel Computer Architecture: A Hardware/Software Approach” by David E. Culler, Jaswinder Pal Singh, and Anoop Gupta. Morgan Kaufmann, 1999. *A treasure filled with details about parallel machines and algorithms. As Mark Hill humorously observes on the jacket, the book contains more information than most research papers.*
- [FLR98] “The Implementation of the Cilk-5 Multithreaded Language” by Matteo Frigo, Charles E. Leiserson, Keith Randall. PLDI '98, Montreal, Canada, June 1998. *Cilk is a lightweight language and runtime for writing parallel programs, and an excellent example of the work-stealing paradigm.*
- [G83] “Using Cache Memory To Reduce Processor-Memory Traffic” by James R. Goodman. ISCA '83, Stockholm, Sweden, June 1983. *The pioneering paper on how to use bus snooping, i.e., paying attention to requests you see on the bus, to build a cache coherence protocol. Goodman's research over many years at Wisconsin is full of cleverness, this being but one example.*
- [M11] “Towards Transparent CPU Scheduling” by Joseph T. Meehan. Doctoral Dissertation at University of Wisconsin—Madison, 2011. *A dissertation that covers a lot of the details of how modern Linux multiprocessor scheduling works. Pretty awesome! But, as co-advisors of Joe's, we may be a bit biased here.*
- [SHW11] “A Primer on Memory Consistency and Cache Coherence” by Daniel J. Sorin, Mark D. Hill, and David A. Wood. Synthesis Lectures in Computer Architecture. Morgan and Claypool Publishers, May 2011. *A definitive overview of memory consistency and multiprocessor caching. Required reading for anyone who likes to know way too much about a given topic.*
- [SA96] “Earliest Eligible Virtual Deadline First: A Flexible and Accurate Mechanism for Proportional Share Resource Allocation” by Ion Stoica and Hussein Abdel-Wahab. Technical Report TR-95-22, Old Dominion University, 1996. *A tech report on this cool scheduling idea, from Ion Stoica, now a professor at U.C. Berkeley and world expert in networking, distributed systems, and many other things.*
- [TTG95] “Evaluating the Performance of Cache-Affinity Scheduling in Shared-Memory Multiprocessors” by Josep Torrellas, Andrew Tucker, Anoop Gupta. Journal of Parallel and Distributed Computing, Volume 24:2, February 1995. *This is not the first paper on the topic, but it has citations to earlier work, and is a more readable and practical paper than some of the earlier queuing-based analysis papers.*

참고문헌

멀티프로세서 스케줄링 / 영문 원문 11쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

[A90] "The Performance of Spin Lock Alternatives for Shared-Memory Multiprocessors" - Thomas E. Anderson. IEEE TPDS Volume 1:1, January 1990. 여러 락 구현 대안의 확장성이 어떻게 좋거나 나쁜지를 다루는 고전적인 논문이다. 시스템과 네트워킹 분야에서 매우 유명한 연구자 Tom Anderson의 논문이다. 그는 아주 훌륭한 운영체제 교재의 저자이기도 하다고 말해 두어야겠다.

[B+10] "An Analysis of Linux Scalability to Many Cores Abstract" - Silas Boyd-Wickizer, Austin T. Clements, Yandong Mao, Aleksey Pesterev, M. Frans Kaashoek, Robert Morris, Nickolai Zeldovich. OSDI '10, Vancouver, Canada, October 2010. Linux를 많은 코어로 확장할 때의 어려움을 다루는 훌륭한 현대적 논문이다.

[CSG99] "Parallel Computer Architecture: A Hardware/Software Approach" - David E. Culler, Jaswinder Pal Singh, and Anoop Gupta. Morgan Kaufmann, 1999. 병렬 컴퓨터와 알고리즘에 관한 세부 내용이 가득한 보물 같은 책이다. Mark Hill이 책 표지에서 재미있게 평했듯, 이 책에는 대부분의 연구 논문보다 더 많은 정보가 담겨 있다.

[FLR98] "The Implementation of the Cilk-5 Multithreaded Language" - Matteo Frigo, Charles E. Leiserson, Keith Randall. PLDI '98, Montreal, Canada, June 1998. Cilk는 병렬 프로그램 작성을 위한 경량 언어이자 런타임이며, 작업 훔치기 패러다임의 훌륭한 예이다.

[G83] "Using Cache Memory To Reduce Processor-Memory Traffic" - James R. Goodman. ISCA '83, Stockholm, Sweden, June 1983. 버스에서 관찰되는 요청에 주의를 기울이는 버스 스누핑을 사용하여 캐시 일관성 프로토콜을 만드는 방법을 제시한 선구적인 논문이다. Goodman이 Wisconsin에서 오랫동안 수행한 연구에는 기발한 아이디어가 가득하며, 이 논문은 그중 한 사례일 뿐이다.

[M11] "Towards Transparent CPU Scheduling" - Joseph T. Meehan. Doctoral Dissertation at University of Wisconsin-Madison, 2011. 현대 Linux의 멀티프로세서 스케줄링이 작동하는 방식의 많은 세부 내용을 다룬 학위 논문이다. 상당히 훌륭하다! 하지만 우리가 Joe의 공동 지도교수였으니 조금 편향되어 있을 수도 있다.

[SHW11] "A Primer on Memory Consistency and Cache Coherence" - Daniel J. Sorin, Mark D. Hill, and David A. Wood. Synthesis Lectures in Computer Architecture. Morgan and Claypool Publishers, May 2011. 메모리 일관성과 멀티프로세서 캐싱을 정리한 결정적인 개론서이다. 어떤 주제에 관해 지나칠 정도로 많이 알고 싶어 하는 사람이라면 반드시 읽어야 한다.

[SA96] "Earliest Eligible Virtual Deadline First: A Flexible and Accurate Mechanism for Proportional Share Resource Allocation" - Ion Stoica and Hussein Abdel-Wahab. Technical Report TR-95-22, Old Dominion University, 1996. 이 멋진 스케줄링 아이디어에 관한 기술 보고서이다. 저자인 Ion Stoica는 현재 U.C. Berkeley 교수이며 네트워킹, 분산 시스템 등 여러 분야의 세계적인 전문가이다.

[TTG95] "Evaluating the Performance of Cache-Affinity Scheduling in Shared-Memory Multiprocessors" - Josep Torrellas, Andrew Tucker, Anoop Gupta. Journal of Parallel and Distributed Computing, Volume 24:2, February 1995. 이 주제를 처음 다룬 논문은 아니지만 이전 연구를 인용하고 있으며, 초기의 일부 큐잉 기반 분석 논문보다 읽기 쉽고 실용적이다.

Homework (Simulation)

In this homework, we'll use `multi.py` to simulate a multi-processor CPU scheduler, and learn about some of its details. Read the related README for more information about the simulator and its options.

Questions

1. To start things off, let's learn how to use the simulator to study how to build an effective multi-processor scheduler. The first simulation will run just one job, which has a run-time of 30, and a working-set size of 200. Run this job (called job 'a' here) on one simulated CPU as follows: `./multi.py -n 1 -L a:30:200`. How long will it take to complete? Turn on the `-c` flag to see a final answer, and the `-t` flag to see a tick-by-tick trace of the job and how it is scheduled.
2. Now increase the cache size so as to make the job's working set (size=200) fit into the cache (which, by default, is size=100); for example, run `./multi.py -n 1 -L a:30:200 -M 300`. Can you predict how fast the job will run once it fits in cache? (hint: remember the key parameter of the `warm_rate`, which is set by the `-r` flag) Check your answer by running with the solve flag (`-c`) enabled.
3. One cool thing about `multi.py` is that you can see more detail about what is going on with different tracing flags. Run the same simulation as above, but this time with `time.left` tracing enabled (`-T`). This flag shows both the job that was scheduled on a CPU at each time step, as well as how much run-time that job has left after each tick has run. What do you notice about how that second column decreases?
4. Now add one more bit of tracing, to show the status of each CPU cache for each job, with the `-C` flag. For each job, each cache will either show a blank space (if the cache is cold for that job) or a 'w' (if the cache is warm for that job). At what point does the cache become warm for job 'a' in this simple example? What happens as you change the `warmup_time` parameter (`-w`) to lower or higher values than the default?
5. At this point, you should have a good idea of how the simulator works for a single job running on a single CPU. But hey, isn't this a multi-processor CPU scheduling chapter? Oh yeah! So let's start working with multiple jobs. Specifically, let's run the following three jobs on a two-CPU system (i.e., type `./multi.py -n 2 -L a:100:100,b:100:50,c:100:50`) Can you predict how long this will take, given a round-robin centralized scheduler? Use `-c` to see if you were right, and then dive down into details with `-t`

과제: 시뮬레이션 (1-5)

멀티프로세서 스케줄링 / 영문 원문 12쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

이 과제에서는 multi.py로 멀티프로세서 CPU 스케줄러를 시뮬레이션하고 몇 가지 세부 사항을 배운다. 시뮬레이터와 옵션에 관한 자세한 내용은 관련 README를 읽어 보자.

질문

1. 먼저 효과적인 멀티프로세서 스케줄러를 만드는 방법을 알아보기 위해 시뮬레이터 사용법을 익혀 보자. 첫 시뮬레이션에서는 실행 시간이 30이고 작업 집합 크기가 200인 작업 하나만 실행한다. 다음과 같이 시뮬레이션된 CPU 하나에서 이 작업, 여기서는 'a'라는 작업을 실행하자: `./multi.py -n 1 -L a:30:200`. 완료하는 데 얼마나 걸릴까? `-c` 플래그를 켜서 최종 답을 확인하고, `-t` 플래그를 켜서 작업과 그 스케줄링 방식을 매 틱마다 추적해 보자.
2. 이제 작업 집합, 즉 크기 200의 작업 집합이 캐시에 들어가도록 캐시 크기를 늘려 보자. 기본 캐시 크기는 100이다. 예를 들어 `./multi.py -n 1 -L a:30:200 -M 300`을 실행하자. 작업 집합이 캐시에 들어가면 작업이 얼마나 빨리 실행될지 예측할 수 있는가? 힌트: `-r` 플래그로 설정하는 핵심 매개변수 `warm_rate`를 기억하자. 해당 표시 플래그 `-c`를 켜고 실행하여 답을 확인하자.
3. multi.py의 좋은 점 하나는 여러 추적 플래그로 내부에서 무슨 일이 일어나는지 더 자세히 볼 수 있다는 것이다. 위와 같은 시뮬레이션을 실행하되 이번에는 `time_left` 추적을 활성화하자(`-T`). 이 플래그는 매 시간 단계에서 CPU에 스케줄링된 작업과, 각 틱의 실행이 끝난 뒤 그 작업에 남은 실행 시간을 모두 보여 준다. 두 번째 열의 값이 감소하는 방식에서 무엇을 발견했는가?
4. 이제 `-C` 플래그로 추적 정보를 하나 더 추가하여 각 작업에 대한 각 CPU 캐시의 상태를 표시하자. 각 작업에 대해 캐시는 해당 작업의 캐시가 차가운 상태이면 공백을, 워밍업된 상태이면 'w'를 표시한다. 이 단순한 예에서 작업 'a'의 캐시는 어느 시점에 워밍업되는가? `warmup_time` 매개변수(`-w`)를 기본값보다 낮거나 높게 바꾸면 어떤 일이 일어나는가?
5. 이제 CPU 하나에서 작업 하나를 실행할 때 시뮬레이터가 어떻게 동작하는지 잘 이해했을 것이다. 그런데 이 장은 멀티프로세서 CPU 스케줄링을 다루는 장 아니었던가? 그렇다! 그러면 이제 여러 작업을 다뤄 보자. 구체적으로 CPU 두 개가 있는 시스템에서 다음 세 작업을 실행하자. 즉, `./multi.py -n 2 -L a:100:100,b:100:50,c:100:50`을 입력하자. 라운드 로빈 중앙 집중식 스케줄러를 사용할 때 얼마나 오래 걸릴지 예측할 수 있는가? `-c`로 예측이 맞았는지 확인한 다음, `-t`로 세부 동작을 살펴보자.

to see a step-by-step and then `-C` to see whether caches got warmed effectively for these jobs. What do you notice?

6. Now we'll apply some explicit controls to study **cache affinity**, as described in the chapter. To do this, you'll need the `-A` flag. This flag can be used to limit which CPUs the scheduler can place a particular job upon. In this case, let's use it to place jobs 'b' and 'c' on CPU 1, while restricting 'a' to CPU 0. This magic is accomplished by typing this `./multi.py -n 2 -L a:100:100,b:100:50,c:100:50 -A a:0,b:1,c:1`; don't forget to turn on various tracing options to see what is really happening! Can you predict how fast this version will run? Why does it do better? Will other combinations of 'a', 'b', and 'c' onto the two processors run faster or slower?
7. One interesting aspect of caching multiprocessors is the opportunity for better-than-expected speed up of jobs when using multiple CPUs (and their caches) as compared to running jobs on a single processor. Specifically, when you run on N CPUs, sometimes you can speed up by more than a factor of N , a situation entitled **super-linear speedup**. To experiment with this, use the job description here (`-L a:100:100,b:100:100,c:100:100`) with a small cache (`-M 50`) to create three jobs. Run this on systems with 1, 2, and 3 CPUs (`-n 1`, `-n 2`, `-n 3`). Now, do the same, but with a larger per-CPU cache of size 100. What do you notice about performance as the number of CPUs scales? Use `-c` to confirm your guesses, and other tracing flags to dive even deeper.
8. One other aspect of the simulator worth studying is the per-CPU scheduling option, the `-p` flag. Run with two CPUs again, and this three job configuration (`-L a:100:100,b:100:50,c:100:50`). How does this option do, as opposed to the hand-controlled affinity limits you put in place above? How does performance change as you alter the 'peek interval' (`-P`) to lower or higher values? How does this per-CPU approach work as the number of CPUs scales?
9. Finally, feel free to just generate random workloads and see if you can predict their performance on different numbers of processors, cache sizes, and scheduling options. If you do this, you'll soon be a **multi-processor scheduling master**, which is a pretty awesome thing to be. Good luck!

과제: 시뮬레이션 (5-9)

멀티프로세서 스케줄링 / 영문 원문 13쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

5. (계속) -t로 단계별 동작을 확인한 다음, -C로 이 작업들에 대해 캐시가 효과적으로 워밍업되었는지 확인하자. 무엇을 발견했는가?
6. 이제 이 장에서 설명한 캐시 친화성을 살펴보기 위해 명시적인 제어를 적용해 보자. 이를 위해 -A 플래그가 필요하다. 이 플래그를 사용하면 스케줄러가 특정 작업을 배치할 수 있는 CPU를 제한할 수 있다. 여기서는 작업 'b'와 'c'를 CPU 1에 배치하고 'a'는 CPU 0으로 제한하자. `./multi.py -n 2 -L a:100:100,b:100:50,c:100:50 -A a:0,b:1,c:1`을 입력하면 된다. 실제로 무슨 일이 일어나는지 확인할 수 있도록 여러 추적 옵션을 켜는 것도 잊지 말자! 이 방식이 얼마나 빨리 실행될지 예측할 수 있는가? 왜 더 좋은 성능을 보이는가? 두 프로세서에 'a', 'b', 'c'를 다르게 조합하여 배치하면 더 빨라질까, 더 느려질까?
7. 캐시를 사용하는 멀티프로세서의 흥미로운 특징 하나는 단일 프로세서에서 작업을 실행할 때와 비교하여 여러 CPU와 그 캐시를 사용하면 예상보다 더 큰 속도 향상을 얻을 수도 있다는 것이다. 구체적으로 CPU N개에서 실행하면 때로는 N배를 넘는 속도 향상을 얻을 수 있다. 이를 초선형 속도 향상이라고 한다. 이를 실험하려면 여기의 작업 명세(-L a:100:100,b:100:100,c:100:100)를 작은 캐시(-M 50)와 함께 사용하여 세 작업을 만들자. CPU가 각각 1개, 2개, 3개인 시스템에서 실행하자(-n 1, -n 2, -n 3). 이제 CPU별 캐시 크기를 100으로 늘리고 똑같이 실행하자. CPU 수가 늘어날 때 성능에 관해 무엇을 발견했는가? -c로 예측을 확인하고 다른 추적 플래그로 더 깊이 살펴보자.
8. 살펴볼 만한 또 다른 시뮬레이터 기능은 CPU별 스케줄링 옵션인 -p 플래그이다. CPU 두 개에서 이 세 작업 구성(-L a:100:100,b:100:50,c:100:50)을 다시 실행하자. 위에서 직접 제어한 친화성 제한과 비교하면 이 옵션의 성능은 어떠한가? '들여다보기 간격'인 peek interval(-P)을 더 낮거나 높은 값으로 바꾸면 성능이 어떻게 달라지는가? CPU 수가 늘어날 때 이러한 CPU별 접근법은 어떻게 작동하는가?
9. 마지막으로 임의의 작업 부하를 자유롭게 생성하고, 프로세서 수와 캐시 크기 및 스케줄링 옵션이 다를 때 그 성능을 예측할 수 있는지 살펴보자. 이렇게 하다 보면 곧 멀티프로세서 스케줄링의 달인이 될 것이다. 꽤 멋진 일이다. 행운을 빈다!